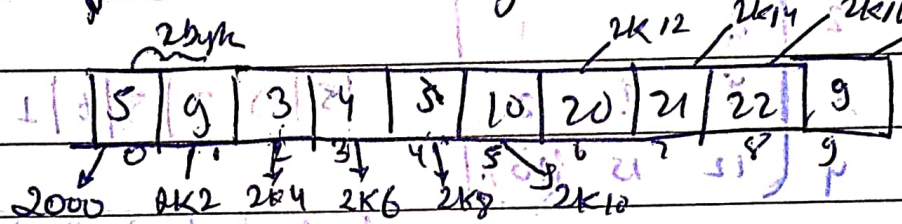
Arrays

Base Address :- ~~from where~~ the element address at that point

int a[10] = { 5, 9, 3, 4, 5, 10, 20, 21, 22, 9 }

Base address of array is 2000 and size of element is Byte 2



location → LOC of A[5] = 2010

Golden rule = no of element skip X byte + Base address

No. of elemen = U.B − L.B + 1

2D Array

int A[4][3] ~~in~~ C language array start from 0 slot it is (5x4) matrix

In memory there is 2D or 3D there is only 1D all things are stored in 1D type of Array.

	1	2	3
1	5	9	3
2	8	6	1
3	4	7	2
4	12	15	10

Row major fashion

5	9	3	8	6	1	4	7	2	12	15	10

5	8	4	12	9	6	7	15	3	1	2	10

Column major fashion

Row major LOC A[4][3]

3 rows are skip

so 1 to 3 we skip then we reach to 4 thus 3-1+1 = (3)

for the column 1 to 2 we skip so

2-1+1 = (2)

3 rows each row 3 elements so 3*3,

after 9 element we reach to 4th row, now we will skip this (2) to reach the A[4][3]

8D total 11 (9+2) we need to skip

Column major

Loc A[3][2]

1 to 2

2-1+1 ⇒ 2 → 2 element skip

↓ i-1+1 = 1 Column skip

1 colum contain 4 element so (4+2) = 6 total

3D Array

assuming first element is A[1][1][1]

if C language then A[0][0][0]

a[10][20][30]

↓ 10 2D Array of 20×30

LOC A[5][6][7]

Row major

4-1+1

↓ 4 2D Array is skipped

= 4×30

6-1+1

5-1+1 = 5 = 5

= 5×30

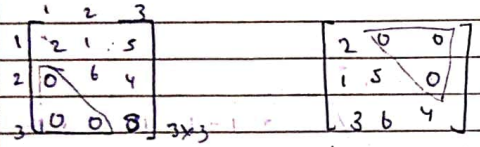
total element is 4×20×30

total element skip = 4×20×30 + 5×30 +

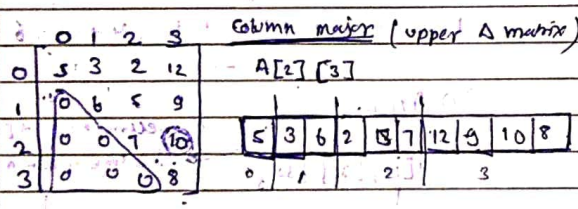
same as column major

(4×20×30 + 5×30 + 5) Size + Base address

Upper and lower Δ matrix



Column major (upper Δ matrix)



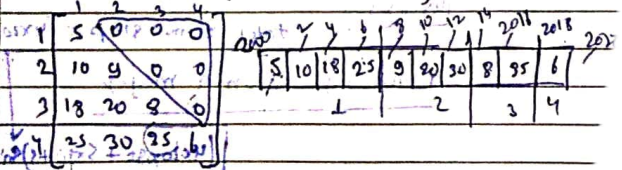
A[2][3]

0 to 2 ⇒ 2+0+1 = 3 → 3(3+1)/2 elements + 3 column skip

1+0+1 = 2 → 2 element skip

so total ⇒ (3(3+1)/2 + 2)

Column major (Lower Δ matrix)



A[x][y] → [(upperbound-1) - lowerbound+1]

A[x][3] → 2-1+1 = 2 → skip the column

3-1+1 = 3 → skip the element

$\frac{2(2+1)}{2}$

*** first we need to search at the end position then we will start from there so total element we skip to search at last is,

$\frac{4(4+1)}{2} = 10$

A[4][3] → upperBound − lowerBound

4-3+1 = 2 → $\frac{2\times(2+1)}{2} = 3$ element

3-3+1 = (1) element

i.e. Row − colum (4-3) = 1

(10 − 3 + 1) = (8) total 8 elemt skipped

And just for Row major (upper △ matrix)

	1	2	3	4
1	1	3	2	12
2	0	4	5	9
3	0	0	7	10
4	0	0	0	8

A[3][4]

4-3+1 = (2) $\frac{2\times(2+1)}{2}$ = (3)

$\frac{2\times(4+1)}{2}$ = (10) total elemnt skipped to Richest at top

one element ⇒ 4-3 = (1)

(10 − 3 + 1)

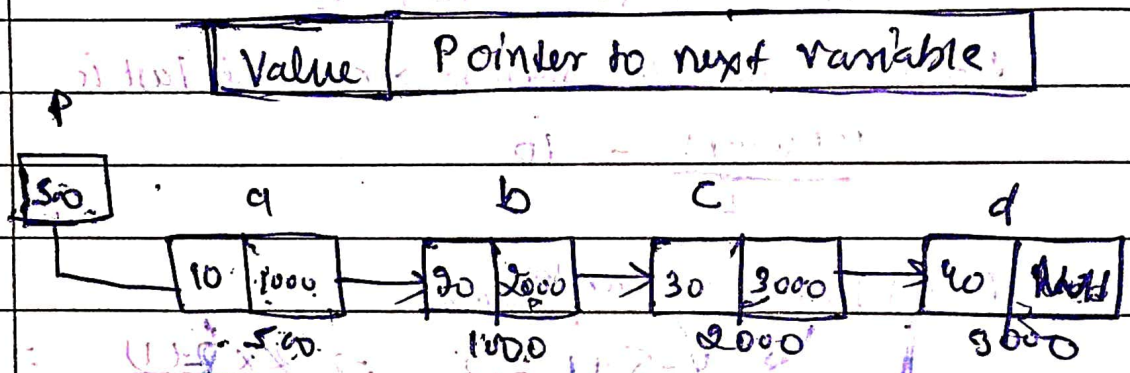
7+1 = (8)

Linked List

multiple list which are linked

int *p; p is pointer to int variable

float *p p is pointer to float variable



printf((*p).val) (output is 10)

printf ((*p).next) (output is 2000)

a ka address 500 hai / b ka address 1000 hai

p = Address of A

p = p → next // 1000

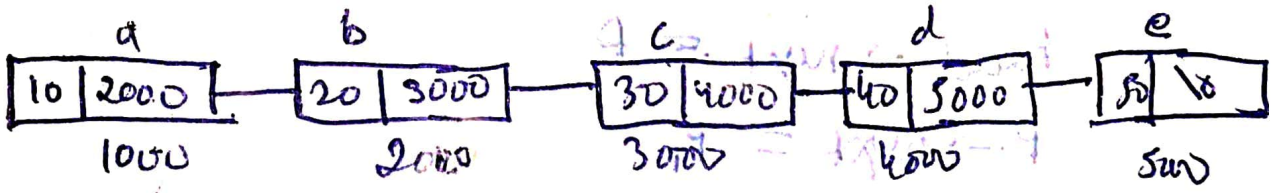
p = p → next // 2000

p = p → next // 3000

p = p → next // Null

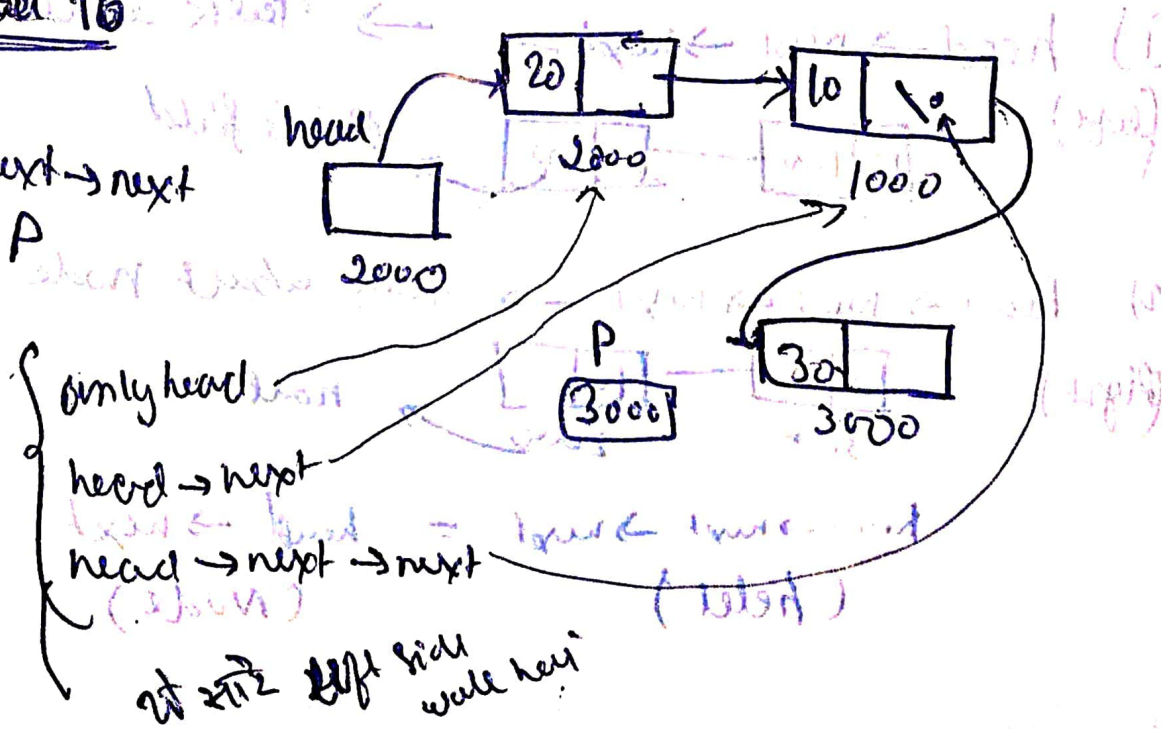
```
List () {
Struct node a = {10, Null};
struct node b = {20, Null};
struct node c = {30, Null};
struct node d = {40, Null};
Struct node e = {50, Null};
  a.next = &b;
  b.next = &c;
  c.next = &d
  d.next = &e
}
```

After writing this Code a linked-List will be made

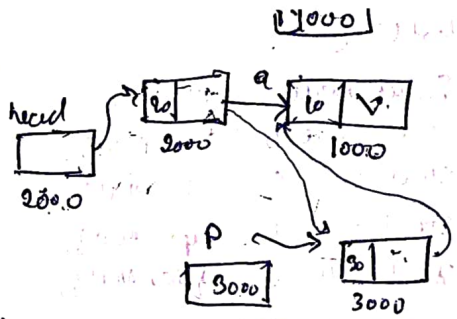


Add P bet 10

head→next→next = P



at the starting



insert p at a

p->val = 30;

If we write :

head->next = p;

But But if we do like this then we will lost access for 10 \ 1000 node

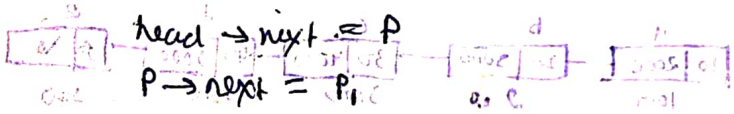
So make a new pointer

Struct node * p1 = NULL

p1 = head->next

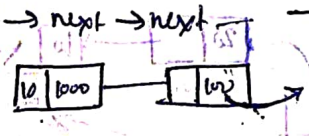
head->next = p

p->next = p1



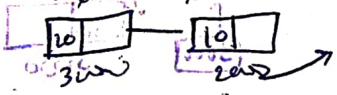
(1) head->next->next → Talk about field

(left)



(2) head->next->next → Talk about node

(Right)



head->next->next = head->next

(field) (Node)

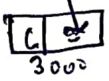


Struct node *p;

P = head → next → next;

// pointer at 3000

P → next → next → next = p;



head → next = p → next

pf (head → next → next → next → next → val);

1000 4000 5000 3000 4000 d

Key → p → next → next → next → next → This will talk the value about field

3000 → 4000 → 5000 → field.

If this is present in left side.

head → next → next = p → next → next → next

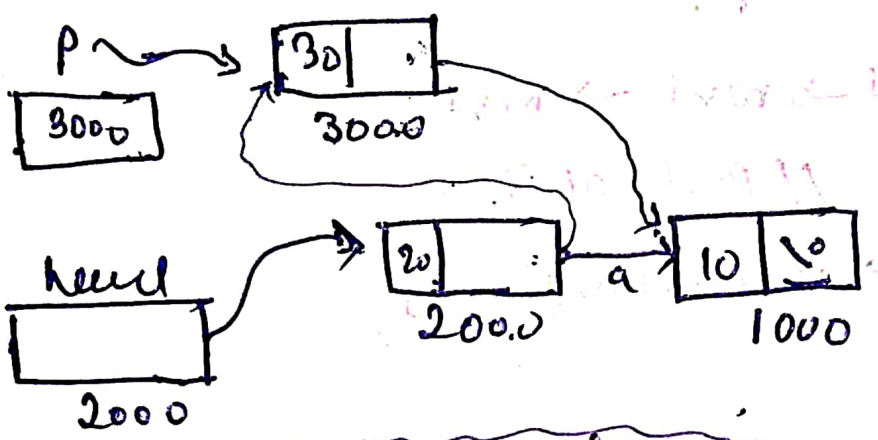
talk about node

talk about node

last "next"

talk about field

If we dont want to make a different pointer then means efficient way to solve this



add p at position a

```
P->next = head->next
head->next = P;
```

If we try to write

```
head->next = P;
P->next = head->next
```

→ means .. node [10|] is last

If list is empty then segmentation fault will occur

If node has two pointers then only we can change the pointers otherwise we will loose the access of after that pointer __/__/____

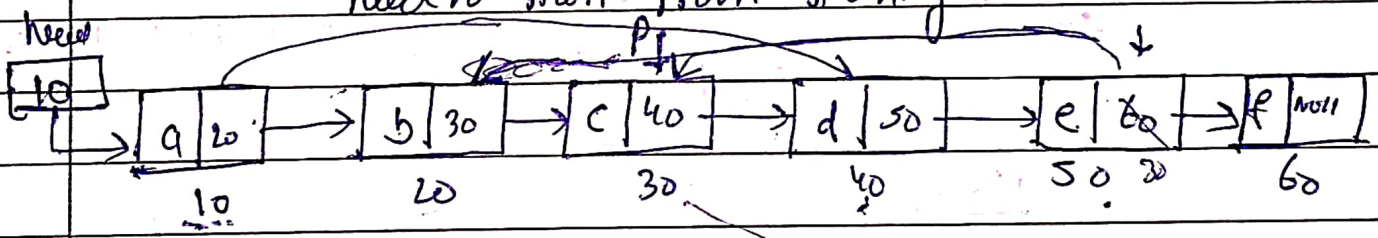
```
P -> val = 10
P -> next = 1000
P = P -> next ;  // 2000
```

but we will loose the access to move back so we will make a new pointer and old one we will move there

```
struct node *p1
p1 = p -> next
p1 = p1 -> next
```

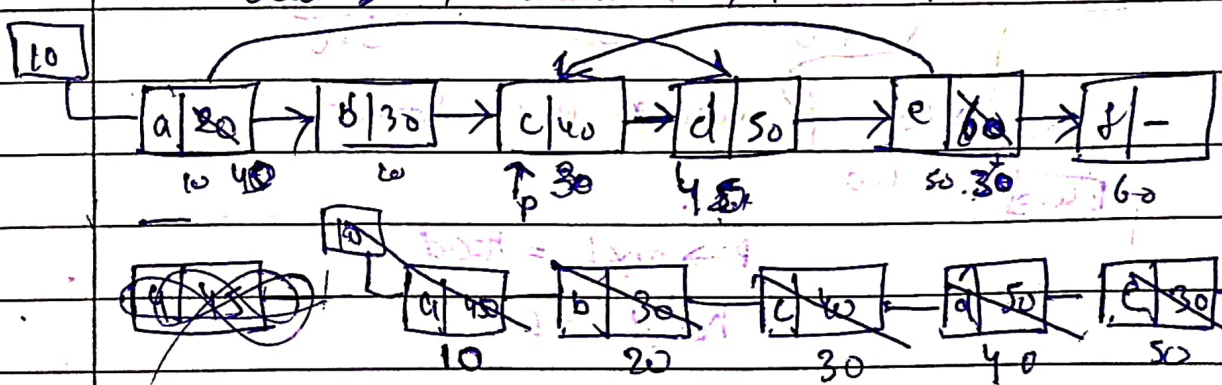
Gate 09 **Note:** to access previous node in Single Linked list we need O(n) time in worst case

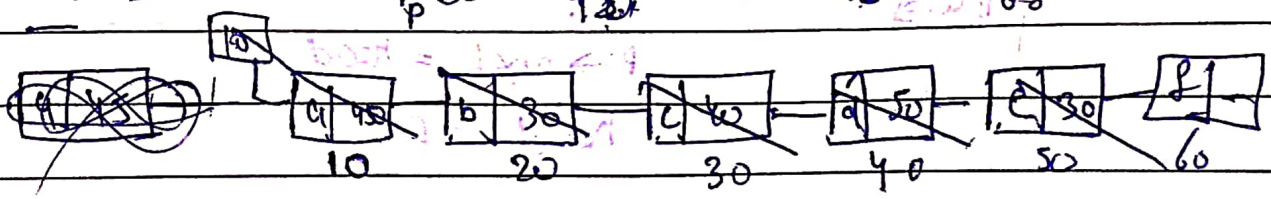
Reason B/c directly we can't move backward we need to start from starting.



Q

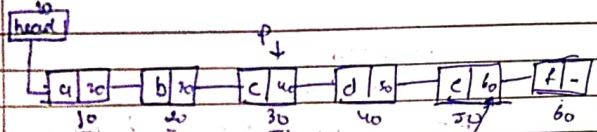
```
struct node *p;
p = head -> next -> next;   // 30
p -> next -> next -> next = p   // 60 = 30
head -> next = p -> next   // 20 = 40
```





__/__/____

Important Rules



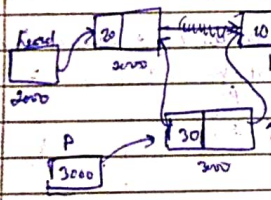
~~head → next → next → next~~

head → next → next = 30

p → next → next → next = p

30 → 40 → 50 → 60 = p last counter next (left side main bracket counter ओर जाएगा)

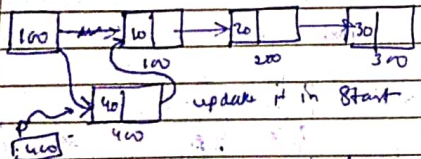
#



p → next = head next ;

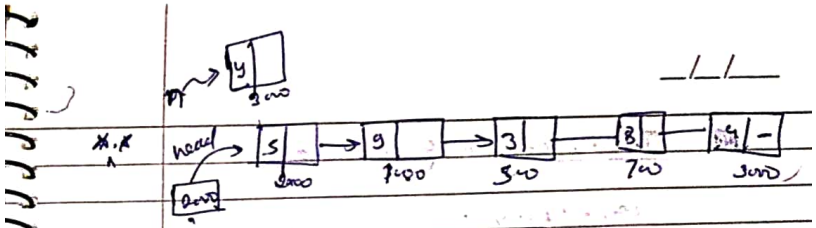
head → next = p;

#



p → next = head

head = p



Traverse the Linked List

```
Traverse(struct node *p) {
    while(p->next != NULL)
    pf(p->val);
    p = p->next
}
```

This will not Print the last value of the node.

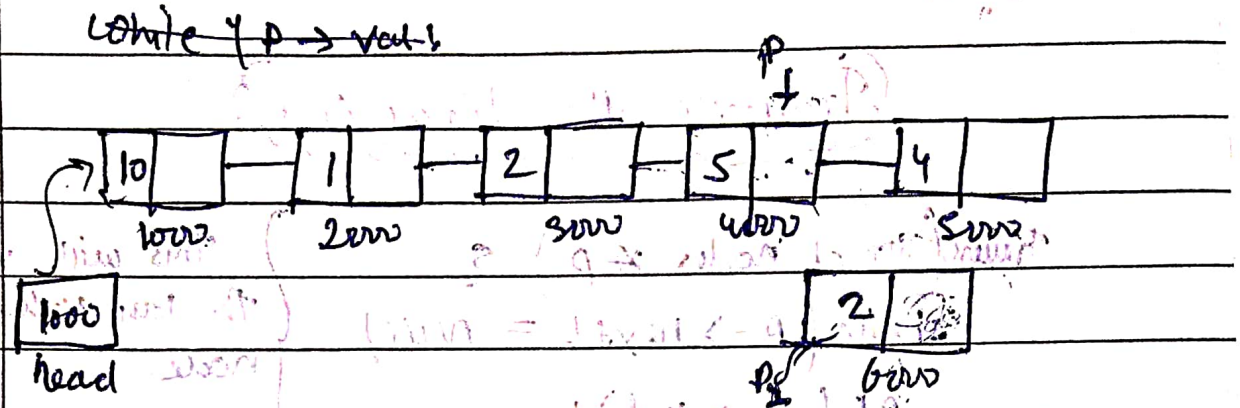
So we need this.

```
Traverse(struct node *p) {
    if(p == NULL)
    pf("List is empty")
    else
    while(p != null) {
    pf(p->val);
    p = p->next
    }
```

Insertion

```
struct node *n = (struct node *) malloc(size of(struct node));
n->val = y
n->next = head
head = n
```

Insertion after a given node:




```
Insert (struct node *p, int x, int y);
while (p->val != 5)
    p = p->next;
struct node *p1 = (struct node*) malloc (sizeof (struct no
```

// we need connect



So

$p_1 \rightarrow next = p \rightarrow next$

$p \rightarrow next = p1$

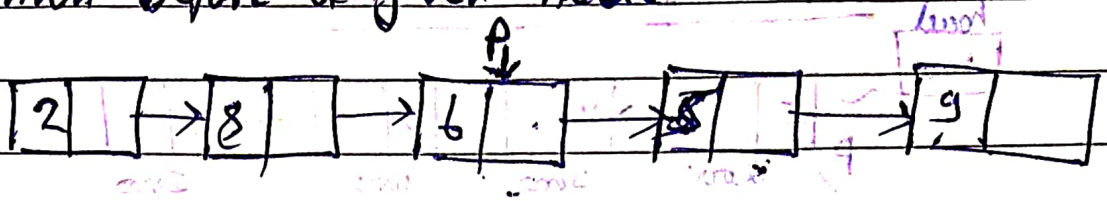
Note: Always check the list is empty or not

↳ for this we will write

```
while (p1 != NULL && p->val != 5)
```

Note: If multiple value are present then the very first value will be updated.

Insertion before a given node



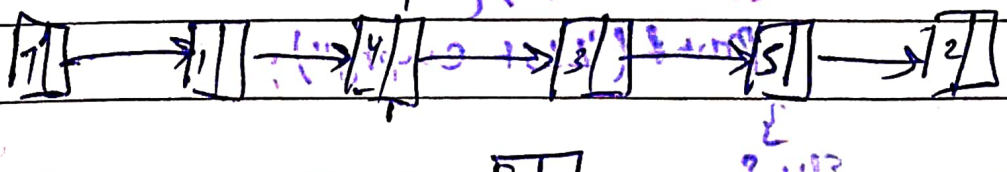



```
while (p->next->value != 5 && p->next != null)
    p = p->next
```

First check this

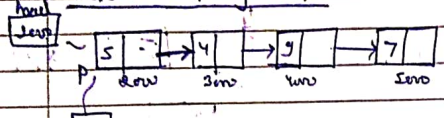
```
if (p->next != null)
struct node *p1 = (struct node *) malloc (sizeof (struct node))
    if (p1 == null) {
        printf ("not found")
    }
Else  p1->val = 3
      p1->next = p->next
      p->next = p1
```

If we want two node before




```
p = p->next->next
while (p->val != 9 && p->next != null) {
    p1 = p1->next
    p = p->next
}
```

Delete starting node



free (head); } don't do this biz due to this
free (p); } the pointer (p) that he is pointing will be free not the pointer. ∞

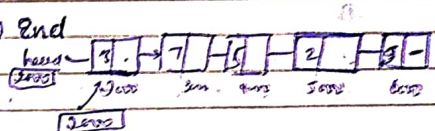
free(p); 4 bytes of memory from 2000 to 2003 is given back to O.S (operating system)

⇒ pointer is pointing to 2000 memory only only the O.S has given its memory to others process

⇒ Indirectly we can say that head and p are pointing to that location which does not exist according to user view.

```
if(p==null){
    printf("List empty");
}
else{
    head = head->next
    free(p);
    p=null;
}
```

Delete at End



```
while (p->next->next != null)
    p = p->next
free (p->next);
p->next = null;
```

This code is not applied for single node.

Before this we will put

```
if (p == null)
    pf ("LLs empty");
else if (p->next == null)
    free (p)
    p = null
```

Deletion of node after node

```
if (p == null || p->next == null)
    pf ("node not found")
Else {
while (p->val != s && p->next != null)
    p = p->next
if (p->val == s)
    p->next = p->next->next
```

18/__/____

The given programme can't check that the node is at last position or not for this we will do little modification

We will do (if (p→next != null))

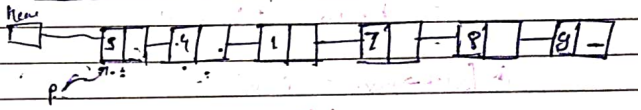
Edit code

```
if (p == null || p→next == null) {
    pf ("node not found")
}
else {
    while (p→val != 5 && p→next != null) {
        p = p→next
    if (p→next != null) {
        p→next = p→next→next
    }
    else { printf ("node not found")
    }
}
```

Mindit

Get code Mein him check kar she hai node ke aage wale node ka ke pointed node. When we will not a get node after the node Then it is will create problem.

Deletion before the node




```
Delet (struct * p)
if (p==null or p->next==null or p->val==5)
    Print ("Node not found")
}
else if (p->next->val==5){
        head = head->next
        free (p)
        p = null
}
else if (p->next->next==null){
    printf ("Node not found")
}
else{
    while (p->next->next->val!=5 && p->next->next->next!=null)
        p = p->next
    if (p->next->next->val==5){
        p->next = p->next->next
        free (p1)
    }
    else{
        Print ("Node not found")
    }
}
```

2 Node की Checking

2 Node से बड़ी लिस्ट जो value 5 नहीं है

3 Node Checking

20/__/____

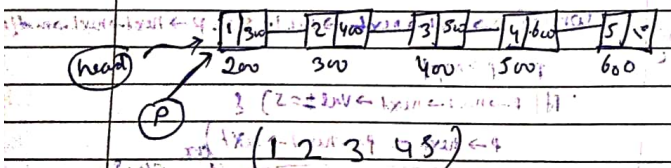
if (!p) कहीं: for Question में इसका मतलब (p != NULL)

(b) (list->next == NULL)
!(list->next)

① while(q)
→ same as while(q != NULL)

Traverse the Linked List by Recursion

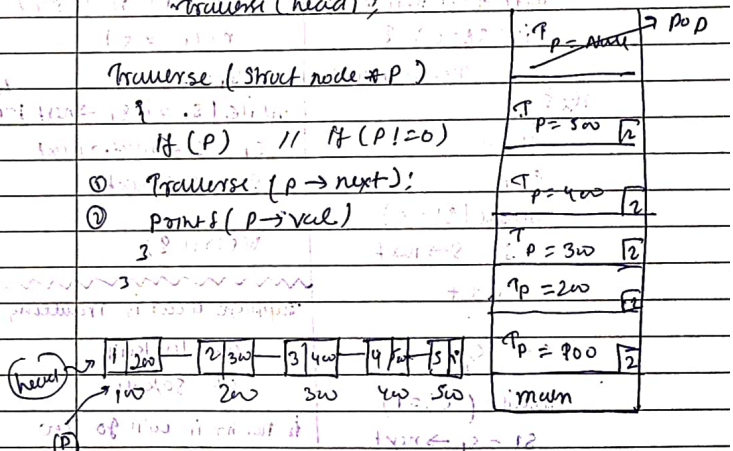
```
void main ;{
Traverse ( head );
Traverse ( struct node *p ) {
  if(p) {      // if(p != 0)
    print (p -> val)
    Traverse ( p -> next );
  }
}
```



~~Reverse the Linkedlist~~

Traverse the Linkedlist in opposite

```
void main ()
{
    Traverse (head);
}
Traverse ( struct node *P )
{
    If (P)  // If (P!=0)
    {
①       Traverse (P -> next);
②       printf ( P -> val)
    }
}
```



22/__/____

How to find mid of linked list




```
s1 mid(struct node *s)
    if(s==0) {
        return NULL;
}
else{
    s1 = s
    c=0;
    while(s!=0)
        s = s->next
        c++
    c = c/2;
    while(c!=1)
        s1 = s1->next
        c--
}
return s1
```

```
(struct node *) mid(struct
        node *s)
    s1 = s
while(s1 && s1->next!=0)
    s1 = s1->next->next
    s = s->next
}
return s;
```

Suppose a car is travelling
at 100 km/h
at 50 km/h
in next hr it will go 100
and 50
then next 300
150
means always half this we
implement this here

T.C = O(n)

23/__/____

Binary Search on Linked List

```
(struct node *) BS (struct node *p, int n)
    if (p == NULL)
        printf ("Linked List is empty")
            return p
    else if (p -> next == NULL)
        if (p -> Val == n)
            return p
        else {
            print ("Node not found")
                return null;
            }
    else {
        struct node *S
        S = mid(p)    // mid function Sophie Bnagar hai
        if (S -> val == n) {
            Printf ("node found");
            return S;
        else if (S -> val > n)
            S -> next = Null
            BS (P, n)
        else
            BS (S -> next, n);
    }
```

24 / /

→ Binary Search can be applied on Sorted array as well as Linked List

→ Only difference is mid finding in array O(1) and Linked List O(n)

→ Hence B.S is not efficient on LLst

→ LS is better than BS for LLS

∵ BC = O(1) (Best case (BC)
WC = O(n) Worst case (WC))

But Binary Search need O(n)

Circular Linked List

We can store 1st node address in place of storing null at the last position in Single Linked List

Advantages

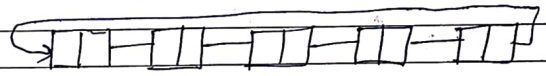
Now we have utilized last node next part & we can access to 2nd node if we are at 3rd (node) node in O(n)

Single Linked List



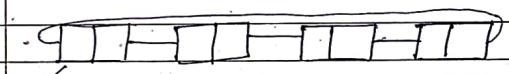
Coming back was not possible

Circular Linked List



(Now we can come back)

Insert node with data x at end of CLL



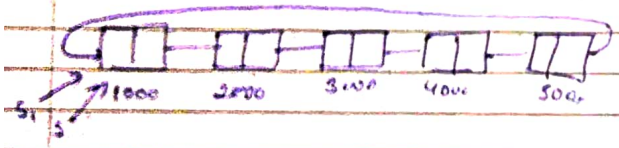



```
struct node *p
p=(struct node *) malloc sizeof (struct node);
if (p==NULL)
    printf ("Memory not available");
    return;
} else
    p->data = x
    p->next = NULL
} making a node that we have to insert
if (s == NULL)
    s = p;
    s->next = s
    return s;
```

```
else {
    S1 = S
    while (S1->next != S) {
        S1 = S1->next
    }
    P->next = S;
    S1->next = P;
}
```

T(n) O(n)

Delete 1st node in Circular Linked List




```
if (S == NULL)
    return S;
S1 = S
while (S1->next != S)
    S1 = S1->next;
S1->next = S->next
free(S)
S = S1->next
```

The previous code will not be executed for single node

for single node we will add small thing

```
if (s->next = s) {
    free(s)
    s = NULL;
}
```

Q) When we can say it is 1st node of Double linked list

Ans

```
if (s->prev == NULL)
    pf("1st node")
```

Q) When we can say it is last node of DLL

```
if (s->next == NULL)
    pf("1st node")
```

Q) How to get address of 2nd last node of DLL

```
while (s->next->next != NULL)
    s = s->next;
    s = s->prev;
```

Q7 How to print elements in reverse order in DLL

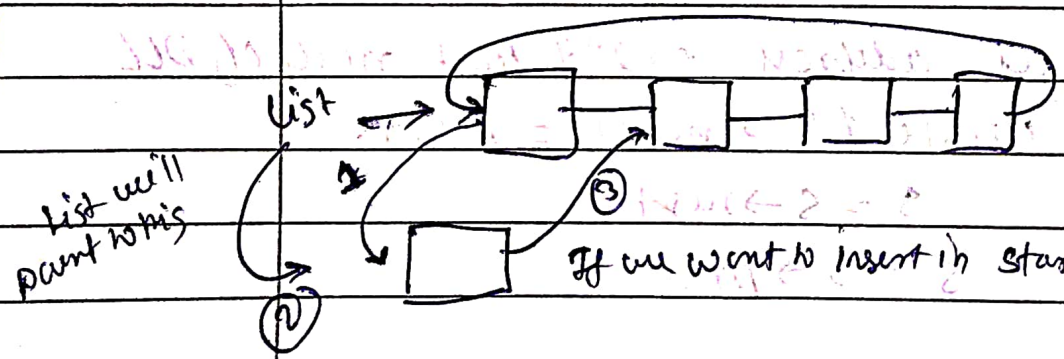
```
while (s -> next != NULL) {
    s = s -> next
}
while (s -> previous != NULL)
    printf ("%d", s -> value);
    s = s -> prev;
}
```

TC(n)

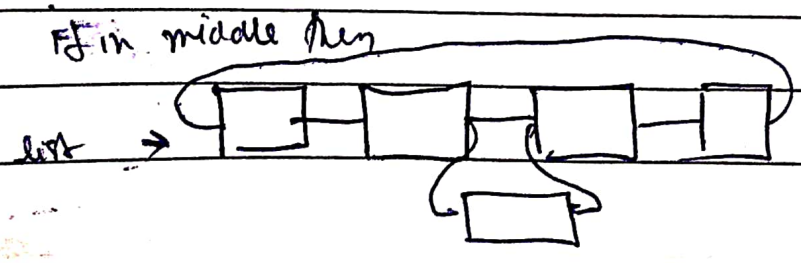
Gate 1987

In a circular linked list organization, insertion of a record involves modification of

1) Two pointer ✓
2) one pointer ✗
3) No pointer ✗
4) Multiple pointer ✗ → this can be true if we insert at starting

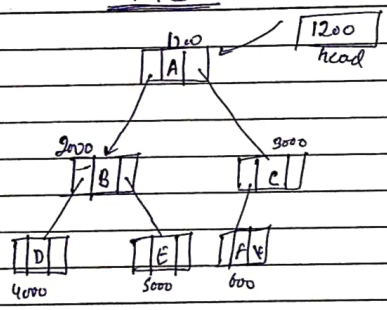


If in middle then



Tree

29/__/____



> Tree is acyclic connected graph
> Any tree having atmost 2 children is called binary tree
> Its like Double Linked list but difference is that in DLL A point to B and vice versa and in Binary tree if A point to B then B never point to A [To avoid cycle]

Linked list given means → pointer to first node given
Tree given means → pointer to root node given

(Here we will call the pointer as root)

Q) when can we say binary tree is empty

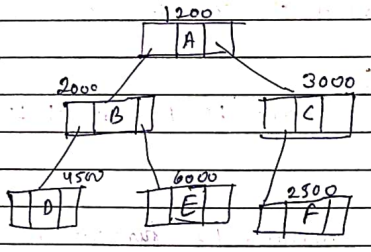
if (root == Null)

Q) When can we say binary tree has only one node

root → lc == Null and root → rc == Null

We can also say DLL is application is BT

Now 12 current lectures in Binary tree.?



Type1

```
int LN(struct node *p)
  if(p==NULL)
     return 0
  else if(p->left==NULL && p->right==NULL)
     return 1
  Else
     L = LN(p->Left)
     R = LN(p->Right)
     T = L+R
     return T
}
```

x

Type2 Instead of x we can also write

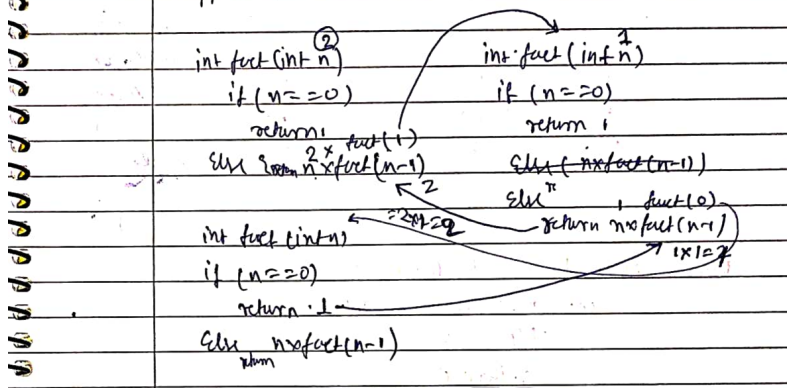
```
Else
     return (LN(p->Left) + LN(p->right))
}
```

Extra How actually the function call works in recursive call

```
int fact(int n)
  if(n==0)
     return 1
  else { return fact(n-1)
```

32/__/____

Suppose I have taken n=2



So overall 2 will be output

Time complexity is we have n node and for each node we call a function once
so O(n)

we can also write in

$$T(n) = 2T(n/2) + c$$

__/__/____

Analysis's

Time complexity

B.C = Ω(n)
A.C = Θ(n)
W.C = O(n)

Space complexity

Ip + extra

B.C = n + log(n)
A.C = n + log(n)
W.C = n + log(n)
↓
O(n)

How to count Num leaf Node.

```
int NL(struct node *P)
    if(P == NULL)
        return 0;
    else if(P->left == NULL && P->right == NULL)
        return 0
    else
        return (1 + NL(P->left) + NL(P->Right))
```

How to count total No of node.

```
int TN(struct node *P)
    if (P == NULL)
        return 0;
    else if (P->left == NULL && P->right == NULL)
        return 0 1
    else
        return(1 + TN(P->Left) + TN(P->Right))
```

__/__/____

If we want 10×LN + 5×InternalNode

```
int Node (Struct node *p)
    if (p==Null)
        return 0;
    else if (p->left==Null && p->right==Null)
        return 10;
    else
        return (5+ Node (P->left) + Node(p->right))
```

Find the (internal node - Leaf node)

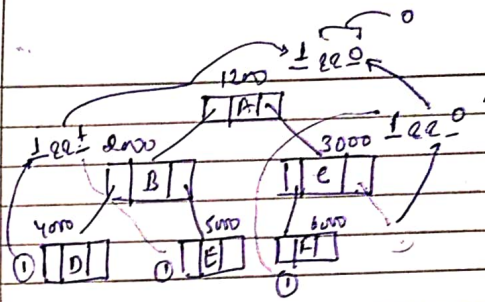
```
int Node (Struct node *p)
    if (p==Null)
        return 0;
    elseif (p->left==Null && p->right==Null)
        return -1;
    else
        return (1 + Node(p->left) + Node(p->right)
```

How to find the height

```
int H(struct node *p)
  if(p==Null)
     return 0
  elsif(p->left==Null && p->right==Null)
     return 0;
  else
     return (1+ max(H(p->left) + H(p->right))
                    a                b
int Max(int a, int b)
     if(a>b)
        return a
     else
        return b
```

How to verify given binary tree is strict or not
[every node has either 0 child or 2 child]

```
int SBT(struct node *p)
  if(p==Null)
     return 1;
  if(p->left==Null && p->right==Null)
     return 1
  if(p->left!=Null && p->right!=Null)
     return (SBT(p->left) && SBT(p->right))
     return 0;
}
```



TC = O(n) Space complexity = O(n)

Write a code when two binary tree are equal

```
Equal (P1, P2)
    if (P1 == NULL && P2 == NULL)
        return 1;
    if (P1 == NULL && P2 != NULL or P2 != NULL && P2 == NULL)
        return 0
    if (P1->val == P2->val)
    return ( P1->left, P2->left) &&
        return Equal( P1->left, P2->left) &&
                Equal(P2->right, P2->right)

Else
    return 0
```

Time Complexity
worst case O(n)
Best case O(1)

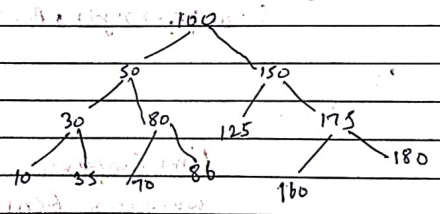
Making the mirror image of Tree

```
mirror (struct node *p)
    if (p == NULL)
        return p
    else if (p -> left == NULL && p -> right == NULL)
        return p
    else if (p -> leftchild != NULL && p -> rightchild != NULL)
        t = p -> leftchild rightchild
        p -> rightchild = (p -> leftchild) mirror
        p -> leftchild = mirror(t)
    return p
```

Time complexity = O(n)

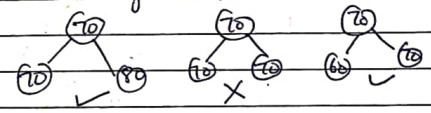
Binary Search Tree

⇒ Binary Search and Binary tree are different



=) Left side small and right side big

Suppose if one side (left side) is ≤ (less then or equal to) then don't apply the equal to function on right side



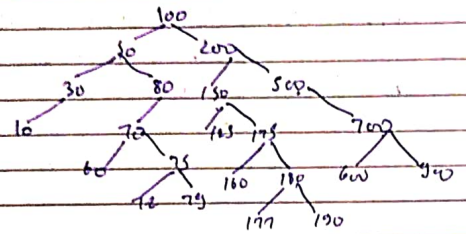
TTS smallest element in Binary Search Tree
⇒ left most element

```
while (temp->left != NULL):
    temp = temp->left
3
return temp
```

Time Complexity

B.C = O(log(n))
W.C = O(n)
A.C = O(log(n))

Searching the element



Code By Q — In which manner you will search 177

100 → 200 → 150 → 175 → 180 → 177

Code Money Money — Time complexity :-

B.C : O(1)

W.C : O(n)

Avg Case : log(n)

Code

Successful Search

	Min element			S.S			LOVE		
	BC	WC	AC	BC	WC	AC	BC	WC	AC
BST	$\Theta(1)$	$\Theta(n)$	$\log(n)$	$\Theta(1)$	$\Theta(n)$	$\log(n)$ / $\Theta(n)$	$\log(n)$	n	$\log(n)$
BT	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$ / $\log(n)$	$\log(n)$	n	$\log(n)$
Min heap	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\log(n)$	$\log(n)$	$\log(n)$
AVL	$\log(n)$	$\log(n)$	$\log(n)$	$\Theta(1)$	$\log(n)$	$\log(n)$	$\log(n)$	$\log(n)$	$\log(n)$

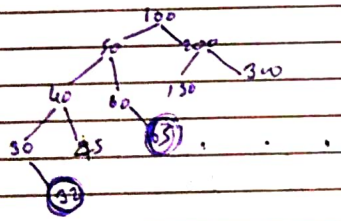
	BC	WC	AC
BST	$\Theta(1)$	$\Theta(n)$	$\Theta(\log(n))$
BT	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Min heap	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
AVL	$\log(n)$	$\log(n)$	$\log(n)$

(USS) unsuccessful search

** while inserting a value in BST it will be implemented as leaf node in middle it will not come

Insertion in BST

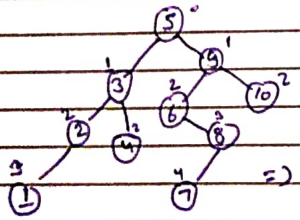
→ Insert 32, 65



How to create BST with n nodes

5, 9, 3, 10, 6, 8, 2, 4, 1, 7

Then will ask how many comparision did you done?



=) 0+1+2+3+4+2+3+1+4
2+3+4

=) 20

Time complexity for ~~Operation~~ BST
→ Creation

BC = $\Theta(n \log n)$

WC = $\Theta(n^2)$

AC = $\Theta(n \log n)$

Deletion in Binary Search tree
→ Leaf Node
→ 1 child
→ 2 child

Condition for 2child If we have to delete 2child node then either replace it with successor or predecessor



Predecessor
one time left then right-right most element

Successor
one time right then left-left most element

Successor and predecessor have at most one child if they will never have 2child

Time complexity for deletion of node

Best case = $O(1)$

Worst case = $O(n)$

Avg case = $O(\log n)$

Time complexity for insertion of node in BST

B.C = $O(\log n)$

W.C = $O(n)$

Time complexity for Creation of BST

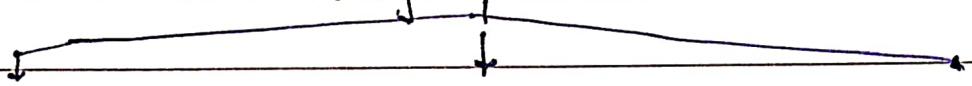
- Balanced = $n \log n$

Unbalanced = n^2

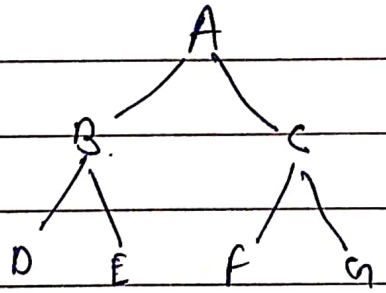
Tree traversal

> Visiting each node of the tree.
> Searching is similar to traversal but Searching should be of worst case

3 way to tree traversal

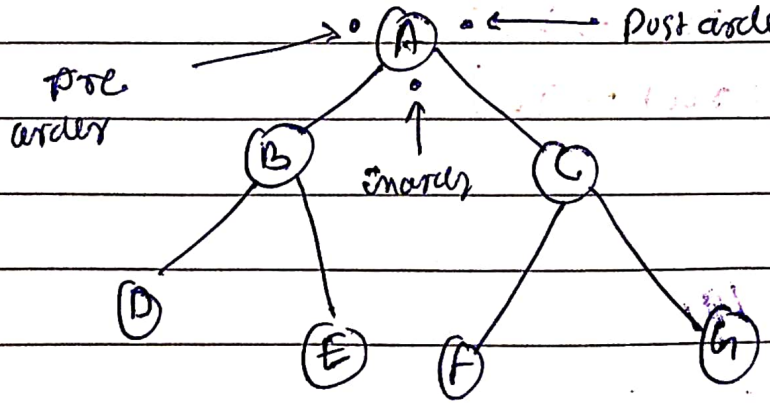


preorder	Inorder	postorder
Root → Left → Right	Left → Root → Right	Left → Right → Root



pre → A B D E C F G
In → D B E A F C G
post → D E B F G C A

preorder (Root first)
postorder (Root Last)

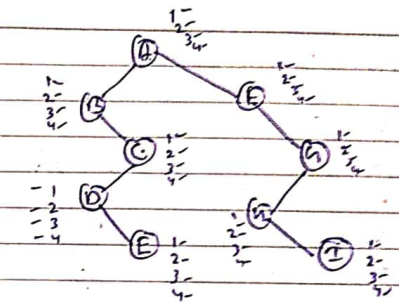


```
preorder(root)
if ( root == NULL)
    return root
elseif ( root != NULL)
  Printf ( Root )
  preorder( root → left)
  preorder (root → Right)
```

Time Complexity	Space Complexity
BC → Ω(n)	BC → O(n)
WC → O(n)	WC → O(n)
AC → Θ(n)	AC → Θ(n)

Double order Traversal

```
Do (root) {
  if (root) {
(1)  Pf ( root → val)
(2)  no ( root → lc)
(3)  pf ( root → val)
(4)  no ( root → rc)
```

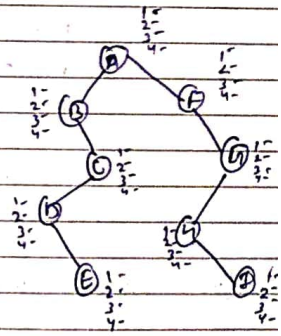


Lecture 12th
22:50

A BBC DDEE CA FFG H H I I G

Q8

```
Do (root)
if (root){
① Do (root → lc)
② Pf (root → val)
③ Do (root → rc)
④ Pf (root → val)
```

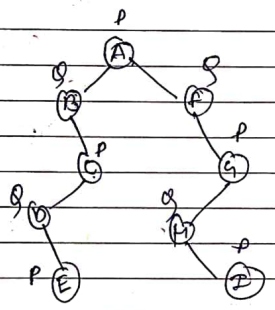


B D E E D C C B A F H I I H G G F A

Indirect Recursion

```
P(root) {
if (root) {
   Pf(root->val)
   Q(root->lc)
   Q(root->rc)
}}

Q(root)
   if(root) {
      P(root->lc)
      Pf(root->val)
      P(root->rc)
```

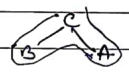


We are starting from P
output: A B C D E F G H I

Note In order will always be in ascending order
Just sort the array that will be
in order

VIII If Post and In order are given then how to find the pre ordered.

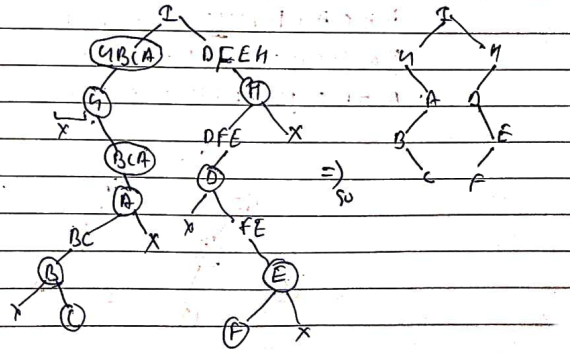
Post : A B C ← Root
In : B C A
L R
pre ??



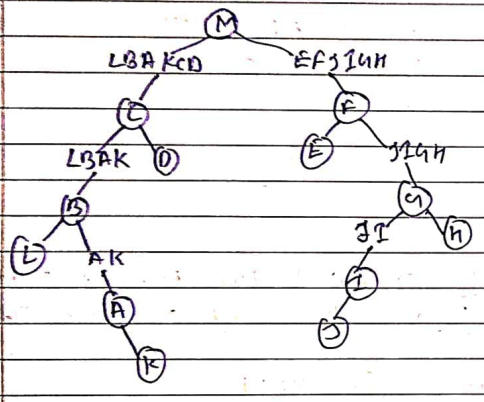
Pre : C B A

Note (Pre) or (Post) se hme Root pata chalta hai
(In) se hme Left or Right pata chalta hai

Pre : IGABCHDEF
In : GBCAIDFEH
L R

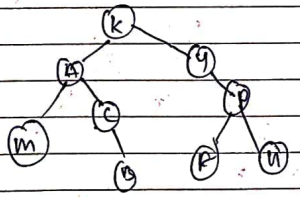


Post : LKABDCEJIHGFM
In : LBAKCDMEFJIGH



Identify pre, In, Post

MBCAFHPYK → Post
KAMCBYPFH → Pre
MABCKYFPH → In



⇒ We are able to make tree when

Pre and In are given ✓

Post and In are given ✓

BST (Binary Search tree)

Pre: 50 40 30 20 10 32 34 36 42 44 60

In: ??

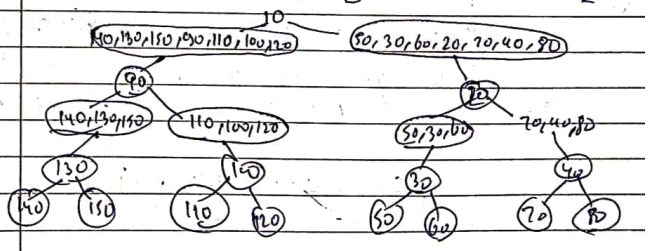
↳ In is always in sorting form

In = 10, 20, 30, 32, 34, 36, 40, 42, 44, 50, 60

After this we can find the Post also bcz we have got pre and In

Min heap given

In: 140, 130, 150, 90, 110, 100, 120, 10, 50, 30, 60, 20, 70, 40, 80



Now we can find any thing

Q How many tree are possible

* ① Pre and Post given → ≥ 0
② Pre and In given ≤ 1
③ post and In given ≤ 1

Q Binary Tree Construction from pre and In order :
or Post and In

BC → $\Theta(n)$
WC → $\Theta(n^2)$ → n nodes + level
AC → $n \log(n)$

Q Pre order Traversal of BST is given find Post order Traversal of BST algo T.C

Best case

Pre order is given
Step1 In order (Sort the pre order) → $\Theta(n \log n)$
Step2 Construct unique BST from Pre & In → $\Theta(n)$
Step3 Find the Post from the tree → $\Theta(n)$

So overall $\Theta(n \log n)$

5/__/__

* If pre and In order given then time required to construct the Binary Search tree = $\Theta(n)$ in Best case

worst case

Step1 Sort In order (Sort the preorder) → $\Theta(n\log n)$

step2 Construct the unique tree (BST) from pre and In order → $\Theta(n\log n)$

Step4 post order traversal → $\Theta(n)$

overall complexity = $\Theta(n\log n)$

for each node log n is required so for n node n log n

Same for the post if post order traversal of BST is given then for BC and WC Time Complexity all thing will be same as pre order

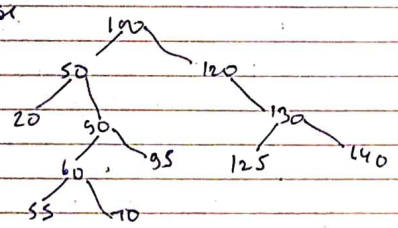
Can we reduce the complexity of n log n by $\Theta(n)$???

Yes

Suppose we have been given a
⇒ pre order Binary search tree

100, 50, 20, 90, 60, 55, 70, 95, 120, 130, 125, 140

Traverse →



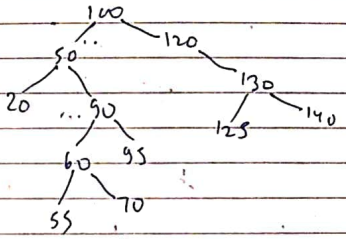
We have moved from left to right and each element is being compared exactly constant time so for n node O(n)

Now we can traverse, post order by O(n)

(1) [This was for if we have given pre BST then find post BST.]

(2) [Now find if we have given post then find the pre BST]

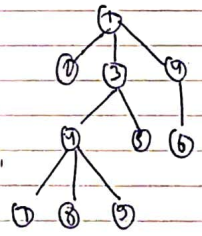
=) 20, 55, 70, 60, 95, 90, 80, 125, 140, 130, 120, 100



If min/max heap: In order traversal dis-given final pre or post order traversal algo TC.

⇒ nlogn

GateP48



Now here we have 3 child (final in order ??)

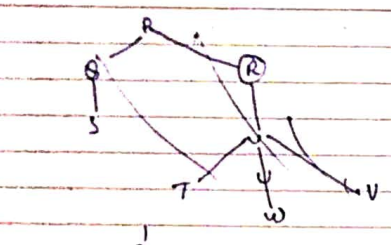
In in order we are confirm that (Left → Root → Right)

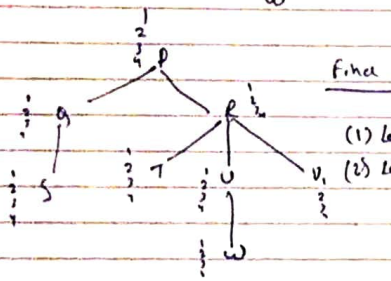
So Left → Root → middle → Right

or Left → middle → Root → Right

Solve it by both which will be option tick that.

Gate 14





Final Encoding

(1) Left — Root — Middle — Right

(2) Left — Middle — Root — Right

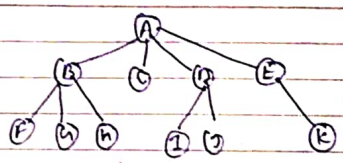
(1) Same is just like Recursion (Left — Root → Middle → Right
1 2 3 4

Q S P T R V W U

(2) Left — middle → Root → Right
1 2 3 4

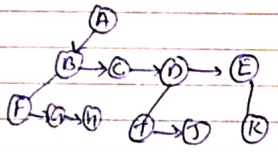
S Q P T W U R V

जब भी Question में न दोनों चाहे कौन middle कौन Left या Right है उसको से नहीं लेना है



To store this binary tree we need 4 pointer but we will try to implement it by binary tree by the help of two pointer

(1) Left most pointer
(2) Node Sibling

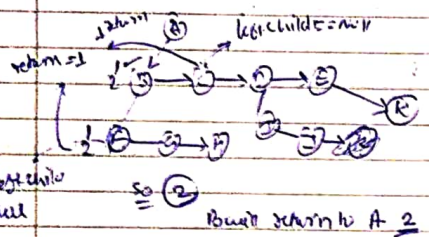


A don't have Sibling
B have Sibling C
C have Sibling D
Hence can be treated as left child

Linking

```
int val = 0
if (tree != NULL) {
    if (tree -> leftmostChild == NULL)
        value = 1
    Else
    1 value = DoSomething (tree -> leftmostChild)
    2 value = value + DoSomething (tree -> rightSibling)
    3
    return (value);
```

Solving this question by example



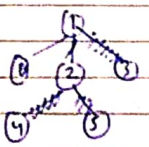
Means here we have not counted G and H
(D, E, K, I, J) this all due to left == null

Counting tree

How many labelled trees of n vertex

n^{n-2} formula

Note



Nodes	Number of trees
3	1
4	2
5	2
2	1

8/1/

So the length of Prüfer code is 4
and length of Node or no of node in tree
= 6

so $\binom{n-2}{6-2} = 4$ right

$n^{n-2} = 6^{6-2} = \boxed{6^4}$ labelled tree of n vertex

(n^{n-2}) is for all tree node not for only binary

↳ not for Binary tree but for Binary tree it should have 2 branch and at most 2 child

Verify $n^{(n-2)}$

$n=1$ $1^{1-1} = 1$ ○

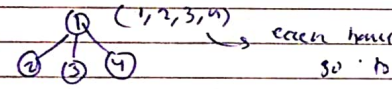
$n=2$ 2^{2-2} ① ↔ ① ↔ ① → ① ① ② ②

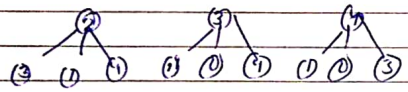
Don't see here this is left this is right here this all are equal

$n=3$ $3^{3-2} = 3$ ① ② ③ ② ① ③ ① ③ ②

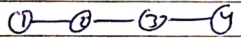
$n=4 \quad n^{n-2} = 4^{4-2} = 4^2$

$= 16$ trees



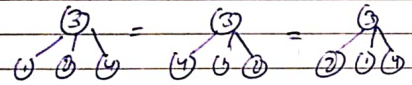


and



$\frac{4!}{2!} = 12$ so $12+4 = 16$

Hint

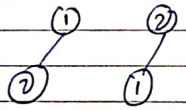


all are same

How many rooted labelled trees on n vertex

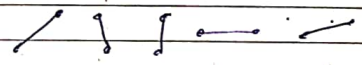
$n^{(n-1)} \times n = \boxed{n^{n-1}}$

$n=2 \quad 2^{2-1} = 2$

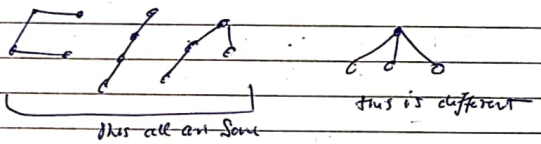


Labelled tree is also known as minimum Spanning tree

How many unlabelled trees of n vertices ?!



This are are same always number

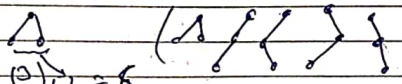


(answer you need to see degree)

How many unlabelled binary tree with n vertex

(here we will consider left and Right child)

Node	No of tree
1	• (1)
2	(2)
3	



$\frac{1}{2} = 5$

How to calculate

internal node = 5

Root node

left	right	
0	4	1 × 14 = 14
1	3	1 × 5 = 5
2	2	2 × 2 = 4
3	1	5 × 1 = 5
4	0	14 × 1 = 14
		42

{by 4 node how many tree you can make that is 14}

$$\frac{2nC_n}{n+1}$$

labelled Binary tree: $\left(\frac{2nC_n}{n+1}\right) n!$

unlabelled Binary tree: $\frac{2nC_n}{n+1}$

labelled tree: n^{n-2}

Rooted labelled tree: n^{n-1}

How many labelled $\frac{2nC_n}{n+1} \times n!$

How many unlabelled binary search tree with n nodes

$$\frac{2nC_n}{n+1}$$

Date 1949

A Complete n-array tree is one in which every node has or n sons. If n is the number of internal nodes of a Complete n-array tree the no of leaves in it is given by



I	L
1	n
2	(n-1) + n
3	(n-2) + n + n.
4	(n-3) + n + n + n
⋮	
n	(n-(n-1)) + (n-1)n

$n-(n-1)+n^2-n$

n^2-n+1

$n(n-1)+1$ So no of leaf node in n array Complete binary tree is

$L = I(n-1)+1$

I → Internal node, n → n array tree, L → leaf Node

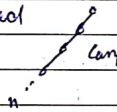
$I+L=n$

AVL tree

Why we come to AVL tree

(1) Binary tree : यहाँ पे कुछ BS element दिये होंगे या फिर किसी भी तरीके से सजाये गए tree mein to if we try to search then we have to search for n nodes

(2) Binary Search Tree ; Here left side skewed by smaller and Right skewed for greater heap when it is skewed



Hence the worst time complexity = O(n)

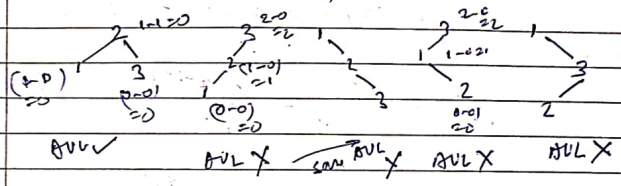
(3) AVL tree (height balanced tree)

$(H_L - H_R = 0, 1, -1)$

$$-1 \leq |H_L - H_R| \leq 1$$

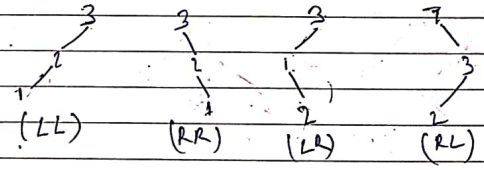
11/__/____

How many BST possible from 3 nodes having labels 1 to 3 (1,2,3)



4 type rotation

> Where the problem has been created from there we need to see

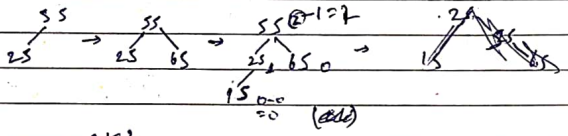


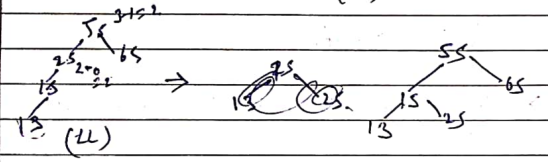
> LL → Right Rotation 1 time
> RR → Left Rotation 1 time
> LR → first left Rotation on left side of Node (n-1) pair and then Right Rotation
> RL → First Right Rotation on Right side of Node (n-1) pair and then Left Rotation

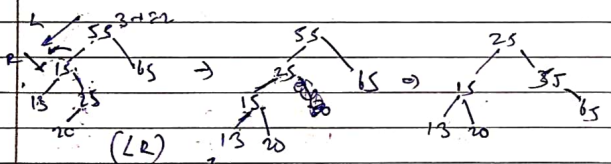
Note: जब Node जहाँ से insert hua की वहाँ से वहाँ से balance factor problem create करें वहा से वहाँ तक देखे कौन सा वहा (LL, LR, RL, RR)

Construct the AVL Tree

55, 25, 65, 15, 13, 20, 37, 51, 16, 53







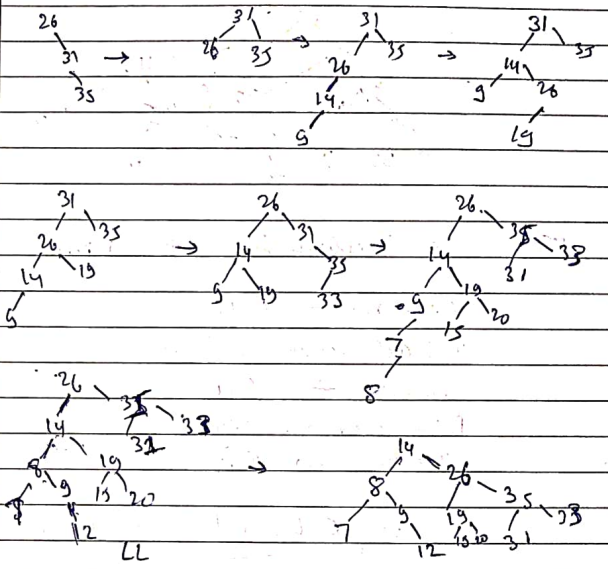
(LR)

(15 → Left = 13
25 → Right of 25, Left of 25)

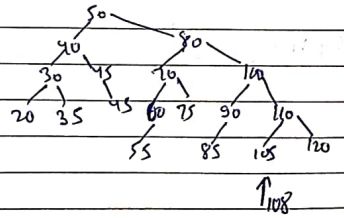
Same by all

12/__/____

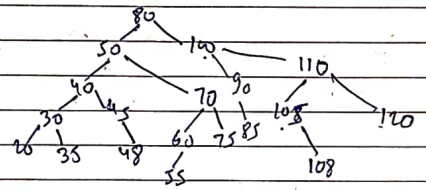
26, 31, 35, 14, 9, 19, 33, 20, 15, 7, 8, 12



Insert 108 in below AVL Tree



Balance factor Check problem at Node 50



Time Complexity for Insertion 1 node

Create node → O(1)
unsuccessful Search → log(n)
Insert → O(1)
Balancefactor → log(n)
Rotation → O(1)

⇒ $O(\log n)$ → for 1 Node

If we are Constructing a AVL Tree Then TC

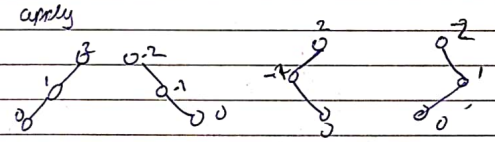
⇒ $O(n \log(n))$

13/1/

BST	AVL Tree
if we have n nodes then maximimum O(n) nodes	if we have n nodes then maximum O(logn) nodes
T.C = n^2	TC = nlogn

Deletation in AVL tree

PROBLEM :- Problem is that when you will delete the node then how to know which Rotation to apply



positive +ve

+ve +ve (Left – Left) Rotation

+ve -ve (Left – Right) Rotation

-ve +ve (Right – Left) Rotation

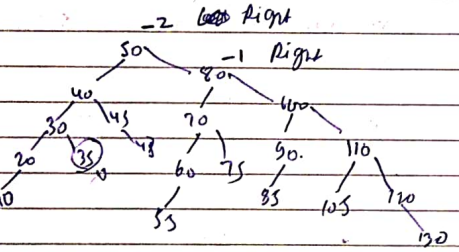
-ve -ve (Right – Right) Rotation

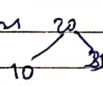
Left +ve

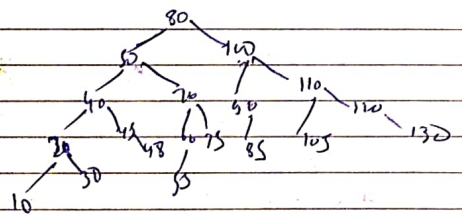
Right -ve

Find AVL Tree after deleting 35 from below.

AVL Tree



Tree becomes  then it will create problem on node 50 (Right Right Rotation)



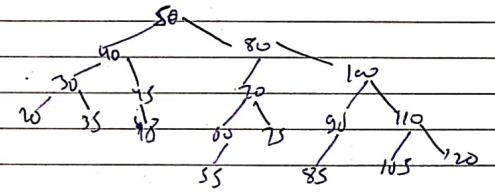
14/__/____

Deleting the Root Node:

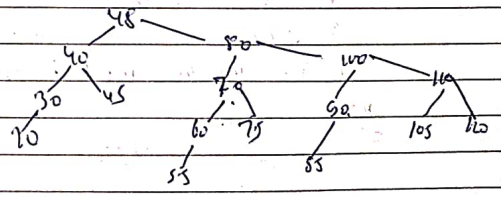
To delete the root node we do 2 proces of BST predecessor and successor

(1) Predecessor (left then extrem Right)
(2) Successor (Right then extrem Left)

Delete 50



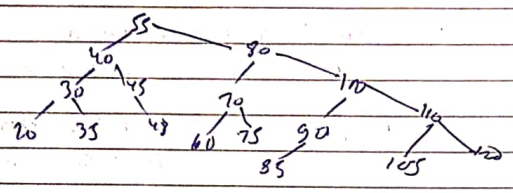
By predecessor (left then extrem Right = 48



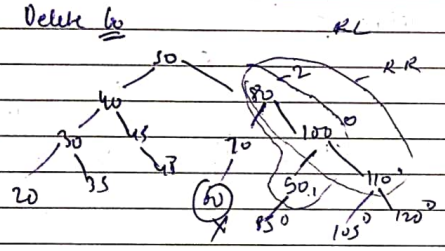
By chance here 0 Rotation is Required

✗ Always we do (left - right) for balance factor

Successor (Right - left - ss)

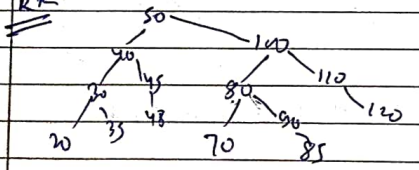


Good Question Delete 60



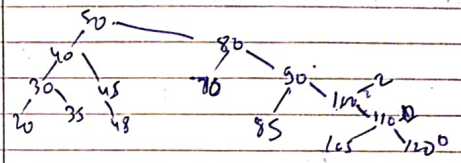
After deleting, the balance factor I have written here after (-2, 0). zero is confusing so we can take '0' as +ve and -ve both and according to that we will match to option

RR



RL

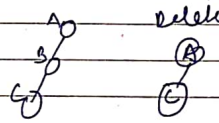
First R then L



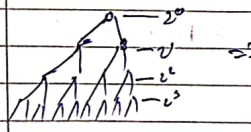
Here we have done RL Rotation but then also we have (Node) -2 case which should not have to case so we do not need to do RL take RR

If we have Single child

Delete B



Min and Max no of nodes in AVL tree of height h



$\Rightarrow 2^0 + 2^1 + 2^2 \cdots 2^h$

$= \boxed{2^{h+1} - 1}$

$\boxed{n = 2^{h+1} - 1}$ Max

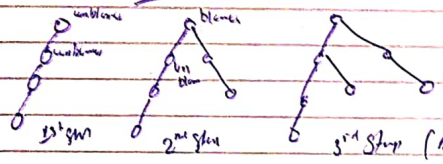
Min

Dont think its diagram will be



like this it can be in any shape only it should be balanced

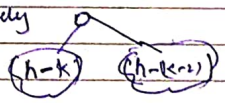
for height 3 Example



1st step 2nd step 3rd step (All Cases)

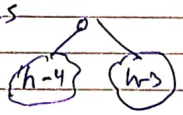
Gate 2006

ultimately



$(k = n-1)$

for height 5



(height - 4)

This will be solved by Recurrence Relation

Gate 2009

$$N(h) = N(h-1) + 1 + N(h-2)$$

$$N(0) = 1$$
$$N(1) = 2$$

10/1/

The binary tree T has n leaf node. The no. of nodes of degree 2 in T is



L = 2 L = 3 L = 4
2 Degree 2 Degree 3 Degree

means

2 Degree Node + 1 = Leaf Node

Gate 2006

for a BST binary tree we know Standard

	Node	BST
K = 1		0
K = 2		
K = 3		

K = 4



3	0	5×1 = 5
2	1	2×1 = 2
1	2	1×2 = 2
0	3	1×5 = 5
		14

means we are doing ~~DFS~~

Gate 2006

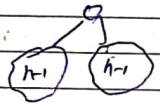
$$T(n) = \sum_{K=1}^{n} T(k-1) * T(n-k)$$

Ques 2015 #

In a binary tree for every node the difference b/w the no of nodes in the left and right subtree is at most 2. If the height of the tree $h > 0$ then the minimum no of nodes in tree is

=) ① 2^{h-1} ② $2^{h-1}+1$ ③ $2^h - 1$ ④ 2^h

By option we can go but lets draw



$N(h)$ min no of nodes in BST of height h which is $|N_L - N_R| \leq 2$

$$N(h) = \underbrace{N(h-1)}_{\text{Left}} + \underbrace{N(h-1)}_{\text{Right}} - 2 + 1$$

Same height Left Right Node → 2 Node are subtracted from Null

Null #

$$N(h) = 2N(h-1) - 1 \qquad N(1) = 2$$

$$2[2N(h-2)-1] - 1$$

$$2^2 N(h-2) - 2 - 1$$

$$2^2[2N(h-3)-1] - 2 - 1$$

$$2^3 N(h-3) - 2^2 - 2^1 - 2^0$$

Binary tree means At most 2 child

17/__/__

$2^K N(h-k) - 2^{k-1} - 2^{k-2} - 2^{k-3} \ldots\ldots 2^0$

$2^K N(h-k) - [2^{K-1}]$

$k = h-1$

$2^K \cdot N(h-(h-1)) - 2^{h-1}$

$2^{h-1} N(h-k+1) - 2^{h-1} + 1$

$2^{h-1} \times 2 - 2^{h-1} + 1$

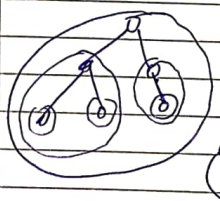
$2^h - 2^{h-1} + 1$

$2^{h-1} \times 2 - 2^{h-1} + 1$

$2^{h-1}[2-1] + 1$

$= \boxed{2^{h-1} + 1}$ Ans

Subtree



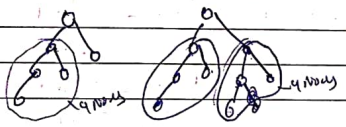
Here all rounded one Subtree

इस Node से बड़ा होता है और उस Node के right child है so not consider करता है even the leaf Node

2014 Consider a rooted n binary tree represented using pointers. The best upper bound on the time required to determine the number of subtree having exactly 4 nodes is $O(n^a \cdot \log^b n)$ find $(a+10b)$

Sol: best upper bound => further worst case complexity

Subtree having exactly 4 nodes

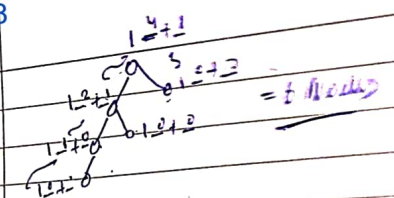


So यहाँ हि 4 node आएगा increment by 1 होगा इसी लिए Count ही आएगा कितना 4 Node वाले subtree है

To Count no of Node

```
ST(root) {
    if (root == NULL)
        return 0;
    else
        return 1 + ST(root->left) + ST(root->right);
}
```

(This will return total no of node)



= 5 Nodes

```
Count = 0
ST (root) {
if (root == NULL)
    return 0;
Else
    L = ST (root → left)
    R = ST (root → Right)
if (1 + L + R == N)
    count ++;
}
return 1 + L + R
```

Every node is visited exactly once and visiting need constant time
so total node is n so $\Theta(n)$

$n^a \log^b n$

$a=1, b=0$ so $a > 1 + 0 \times b = 1$